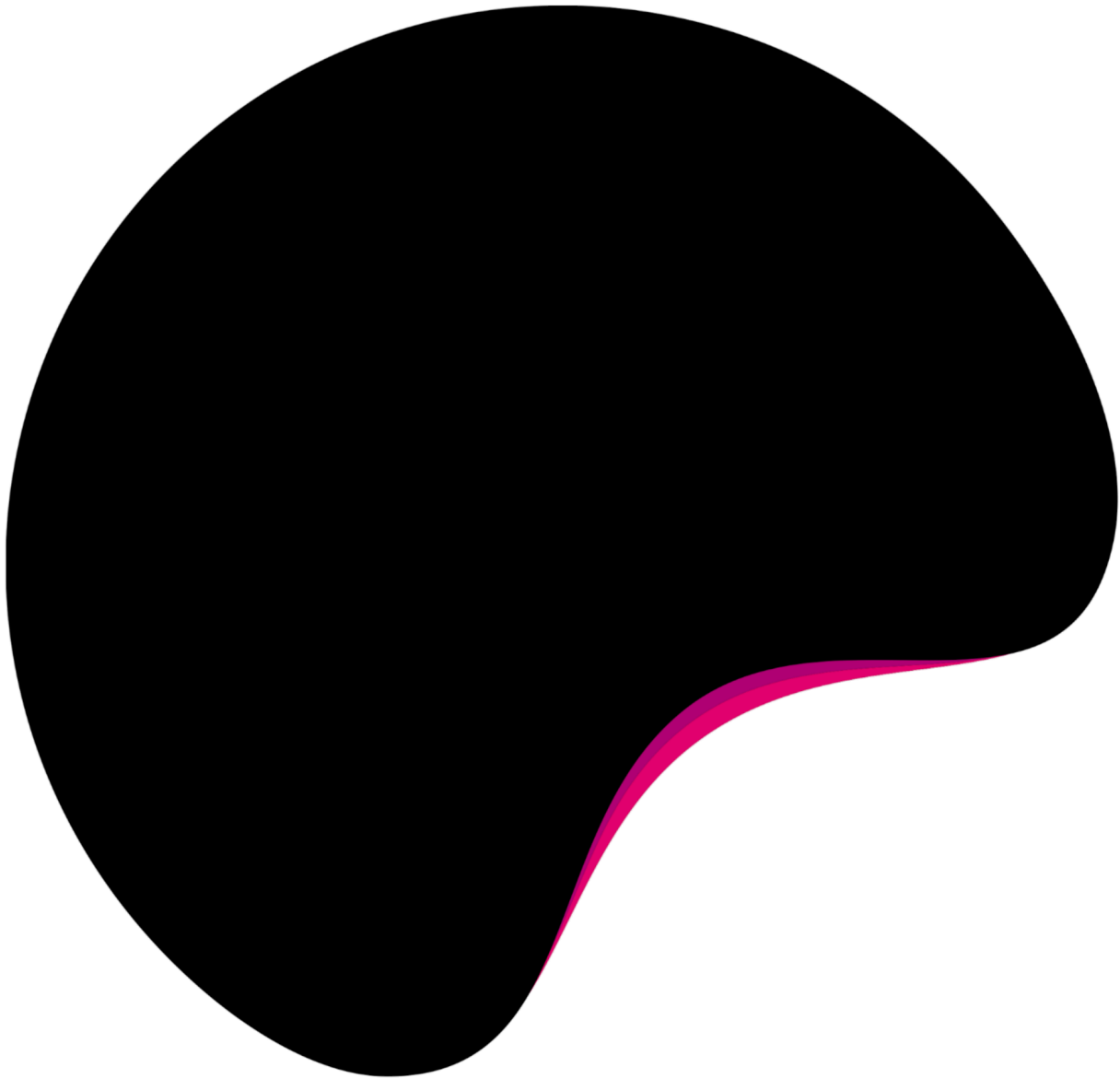




GlowCore

SE Project Documentation

Version: 00.00

Date: 13.03.2026

Git Version: 7f13327

Project Team: Cedric Cathomas
Yoris Kucera
Dominik Wyss
Nathanael Fässler

Project Advisor: Thomas Bocek

School of Computer Science
OST Eastern Switzerland University of Applied Sciences

Contents

I	Management Summary	1
II	Product Documentation	3
1	Requirements	4
1.1	Functional Requirements	4
1.2	Non-Functional Requirements	4
2	Domain Analysis	5
3	Architecture	6
4	Quality Measures	7
4.1	Working Environment	7
4.2	Testing	7
III	Project Documentation	8
5	Initial Project Proposal	9
5.1	Team Members	9
5.2	Availabilities	9
5.3	Vision	9
5.4	Proposed Realisation	10
6	Project Plan	11
6.1	Resources	11
6.1.1	People	11
6.1.1.1	Project Management Roles	11
6.1.1.2	Developer Roles	12
6.1.2	Time	12
6.1.3	Cost	12
6.2	Organization	12
6.2.1	Meetings	12
6.2.2	Planning Tools	13
6.3	Schedule	14
6.3.1	Work Items	14
6.3.2	Long Term Plan	14
6.3.2.1	Milestones	14
6.3.2.2	Epics	16
6.3.2.3	Initial Backlog & MVP	17
6.3.3	Short Term Plan	18
6.3.3.1	Sprint 1 (Sprint Zero) 20.02.2026 - 06.03.2026	19
6.4	Risk Management	19
6.4.1	Risk Matrix	20
6.4.2	Risk Matrix (Mitigated)	21
7	Time Tracking Report	22
7.1	Time Tracking	22
7.2	Time Statistics	22
8	Personal Reports	23
9	Meeting Minutes	24
9.1	Weekly Scrum / 28.02.2026	24
9.1.1	Agenda	24
9.1.2	Todos	25
9.2	Weekly Scrum and Sprint Retrospective / 06.03.2026	26
9.2.1	Agenda	26
9.2.2	Sprint 1 Retrospective	26
9.2.2.1	What Went Well	26
9.2.2.2	What Didn't Go Well	27

9.2.2.3 Improvements	27
9.2.3 Todos	27
IV Bibliography	29
V List of Figures	30
VI List of Tables	31
VII Code Listings	32

Part I

Management Summary

Management Summary

Part II

Product Documentation

1 Requirements

1.1 Functional Requirements

1.2 Non-Functional Requirements

2 Domain Analysis

3 Architecture

4 Quality Measures

4.1 Working Environment

4.2 Testing

Part III

Project Documentation

5 Initial Project Proposal

Project name: GlowCore

Date: 19.12.2025

5.1 Team Members

1. Cedric Cathomas (Cedric.Cathomas@ost.ch)
2. Dominik Wyss (Dominik.Wyss1@ost.ch)
3. Nathanael Fässler (Nathanael.Faessler@ost.ch)
4. Yoris Kucera (Yoris.Kucera@ost.ch)

5.2 Availabilities

Time slot	Mon	Tue	Wed	Thu	Fri
08h00–09h00	-	(XR)	-	-	(XO)
09h00–10h00	-	(XR)	-	-	(XO)
10h00–11h00	(XR)	-	-	-	(XO)
11h00–12h00	(XR)	-	-	-	(XO)
12h00–13h00	-	-	-	-	(XO)
13h00–14h00	XR	-	-	-	(XO)
14h00–15h00	XR	-	-	-	(XO)
15h00–16h00	(XR)	XR	-	-	(XO)
16h00–17h00	(XR)	XR	-	-	(XO)
17h00–18h00	-	(XR)	-	-	(XO)
18h00–19h00	-	(XR)	-	-	-

Table 1: Team availability for review meetings

5.3 Vision

GlowCore is a singleplayer progression game focused on resource gathering and gradual world expansion. At the start, the player can only access a small, limited area of the game world, while the rest remains hidden and unreachable. At the center of the world is a special entity that can be upgraded using collected materials. Each upgrade expands the accessible area, revealing new parts of the world and enabling further progression. The core gameplay loop is built around exploring newly unlocked areas, gathering resources, and using them to continuously improve the central entity and push the boundaries of the world outward.

As the world expands, additional gameplay systems deepen the experience. The player can obtain and use different materials, interact with tools, craft items, and face enemies within the expanding world. More advanced mechanics such as machines for processing resources, farming systems, and specialized equipment can be introduced over time to increase complexity and strategic depth.

5.4 Proposed Realisation

The project will be implemented using Unity as the primary game engine, using its integrated systems for rendering, physics, input handling, animation, audio, and crossplatform deployment. Development will be done in C# using Unity's component-based architecture (GameObjects, MonoBehaviour, and ScriptableObjects) to structure gameplay logic and systems. Unity's built in physics engine and animation system (Animator) will be used to realise core gameplay and user interaction. Visual assets will be rendered using Unity's Universal Render Pipeline (URP). Individual systems will be tested for robustness using unit tests by method of Unity's own testing framework, based on NUnit. Version control will be handled with Git. The project will be developed within the Unity Editor.

6 Project Plan

6.1 Resources

6.1.1 People

The project team consists of 4 students. Each member contributes approximately 8.6 hours per week and is responsible for tasks of their defined roles.

6.1.1.1 Project Management Roles

Role	Description	Names
Product Owner	• Define product vision • Manage backlog • Prioritize features • Validate requirements	Nathanael Fässler
Scrum Master	• Facilitate meetings • Remove blockers • Ensure Scrum process • Support team workflow	Dominik Wyss
Project Manager	• Schedule planning • Resource management • Risk control • Progress reporting	Cedric Cathomas
DevOps Engineer	• CI/CD maintenance • Build automation • Deployment support • Infrastructure management	Yoris Kucera

Table 2: Project Management Roles

6.1.1.2 Developer Roles

Role	Description	Names
Lead Developer	• Overall technical coordination • Code quality control • Architecture decisions • Integration management	Yoris Kucera
Developer	• Gameplay implementation • Bug fixing • Feature development • System integration • Writing Test Cases	Dominik Wyss Yoris Kucera Nathanael Fässler Cedric Cathomas
Art Director / 3D Artist	• Asset creation • Scene design • Visual consistency • Asset optimization	Nathanael Fässler
UI Architect	• UI/UX design • Menu systems • HUD implementation • User interaction flow	Cedric Cathomas
Graphics Programmer	• Shader development • Particle effects • Performance optimization • Visual effects integration	Yoris Kucera
Audio Engineer	• Sound design • Audio integration • Music management • Audio balancing	Dominik Wyss
Level Designer	• Level layout • Gameplay balancing • Difficulty tuning • Playtesting feedback	Nathanael Fässler Cedric Cathomas

Table 3: Developer Roles

6.1.2 Time

The project runs from 16.02.2026 to 12.06.2026 with an estimated effort of 120 hours per person, resulting in a total workload of 480 hours, structured in milestones and sprints.

6.1.3 Cost

Most required tools and platform are freely available through free or student licenses. We plan on publishing GlowCore on steam for which steam charges a 100 US-Dollar publishing fee. The cost will be shared by the members.

6.2 Organization

6.2.1 Meetings

The project meetings follow mainly the Scrum standard. With weekly Scrum meetings replacing daily stand-ups and biweekly sprint planning including retrospectives. Also the Regular advisor meetings to present and discuss our results so far.

Type	Todos	Frequency
Sprint Review (Advisor)	• Present sprint results • Demonstrate features • Discuss open issues • Collect advisor feedback • Document decisions	Every 2 weeks Tuesday 15:00 - 16:00
Weekly Scrum	• Report progress • Define next steps • Identify blockers • Update tasks • Support team members	Weekly Friday 14:00 - 15:00
Sprint Retrospective	• Analyze sprint process • Identify problems • Define improvements • Document lessons learned	Every 2 weeks Friday 15:00 - 16:00
Sprint Planning	• Define sprint goals • Select backlog items • Estimate effort • Assign tasks • Update schedule	Every 2 weeks Friday 16:00 - 17:00 (after retro)

Table 4: Planned Meetings Overview

6.2.2 Planning Tools

Tool	Use Case
GitHub	• Version control • GitHub Actions (CI/CD) • Pull requests • GitHub notifications via Discord
Typst	• Project documentation • PDF generation
Microsoft Teams	• File sharing • Communication
Jira	• Task management • Sprint planning • Backlog tracking • Issue tracking • Time tracking
Miro	• Brainstorming • Diagrams • Project planning
Discord	• Online meetings • Informal communication • Quick coordination • GitHub activity notifications

Table 5: Planning Tools

6.3 Schedule

The project follows the OST internal SCRUM+ framework for its overall organization. SCRUM+ combines elements of Scrum with the Rational Unified Process (RUP). Long-term planning is based on the RUP approach, while short-term development is organized using Scrum with two-week iterations.

At the start of each iteration, we estimate all tickets with storypoints. The story **GC-54 Make Player** is the reference story which is worth 5 story points.

6.3.1 Work Items

We work with 5 types of Work Items

Name	Description
Epic	A group of features that we want to implement.
Story	Describes a feature that is directly related to the game
Task	Everything that isn't directly related to the game
Bug	Bugs are created when a bug is found and fixed in an unplanned matter.
Documentation	Work related to the documentation

Table 6: Work Items

6.3.2 Long Term Plan

The Long Term Planning is separated into two categories:

- Milestones: Serve as checkpoint for releases and review meetings.
- Epics: Provide the structure to organize the stories.

6.3.2.1 Milestones

The milestones for the review meetings and for the releases are interleaved and independent.

Nr	Milestone Name	Date	Description
M01	Review 1	16.02.2026	Initial Project Setup
M02	Review 2	11.03.2026	Requirements
M03	Review 3	25.03.2026	End of Elaboration
M04	Review 4	08.04.2026	Quality
M05	Review 5	29.04.2026	Architecture
M06	Project Presentation	12.06.2026	Final Presentation to SEProject-Advisors and fellow students. Completion of SEProject.
M07	Pre-Alpha Release	20.03.2026	First playable prototype is ready. Core gameplay loops are implemented. The player can chop trees, pick up wood and expand the GlowCore.
M08	Alpha Release	10.04.2026	MVP is done and working. Core gameplay Mechanics are implemented. Player can craft, build, expand the map and he has an inventory. The player can save his savestate.
M09	Beta Release	15.05.2026	More Biomes and Materials are added. There is a combat system with enemies and peaceful mobs. There is a title screen and a settings menu. Automation mechanics are implemented. There are more GlowCore Phases.
M10	Official Release and Final Submission	05.06.2026	The game is polished and has no mayor bugs. The game has a beginning and an ending.

Table 7: Milestones

6.3.2.2 Epics

Epic Name	Description/Requirements
Documentation	This epic includes all documentation related work.
Project Setup	This epic contains all work that is done in the Inception-Phase.
Core Gameplay Loop Mechanics	- The player can chop down trees, collect wood and feed it into the GlowCore. - The GlowCore levels up and expands the map. - The player is restricted to the unlocked map. The grid system is implemented.
Inventory, Crafting, Building	- Inventory: The player has an inventory and can store a limited amount of items. He can have items in his hands and in his backpack(or similar). - Crafting: The player can craft items to other items on different workstations. - Building: The player can place objects onto the world-grid.
Graphics	- The Artstyle of the game ist set. - The 3D Models follow a consistent Artstyle. - A Shader is implementet.
Game Framework & Completeness	This epic contains everything that makes a game complete and feel like a game. - Intro: When the Game starts the GlowCore Logo is displayed. After the title screen a Cutscene is played that leads into the game. - Ending: A ending of the game is defined. When the player finishes the game, a Cutscene is played and the credits roll. - UI: A Title Screen is available. In-Game there is a Debug Screen, and a Settings Menu. - Gamedata Saving: The player can save his progress.
Big World & GlowCore Phases	- World building: The game has a complete world to play in. - GlowCore Phases: The GlowCore has multiple phases, all of which need different materials to level up. The GlowCore changes visually with every Level.
Biomes & Materials	- The world has multiptle biomes: e.g. Forest, Grassland, Mountain, Volcano, Jungle etc. - Each biome has unique ressources. - There are Caves which you can enter.
Combat	- Enemies can spawn and fight with the player. - The player can take damage and can die. - A respawn mechanic is set up. - There are peacuful mobs.
Resource Automation	- The Player can replant trees. - The Player can craft machines that automatically gather resources.
Playtesting & Release	- Playtesting: The game is testet by players that play it for the first time (blind test). - Release: The game is released on steam on its own steam page and has a trailer, screenshots and a description.

Table 8: Epics

The following image shows the Timeline in Jira as of 04.03.2026.

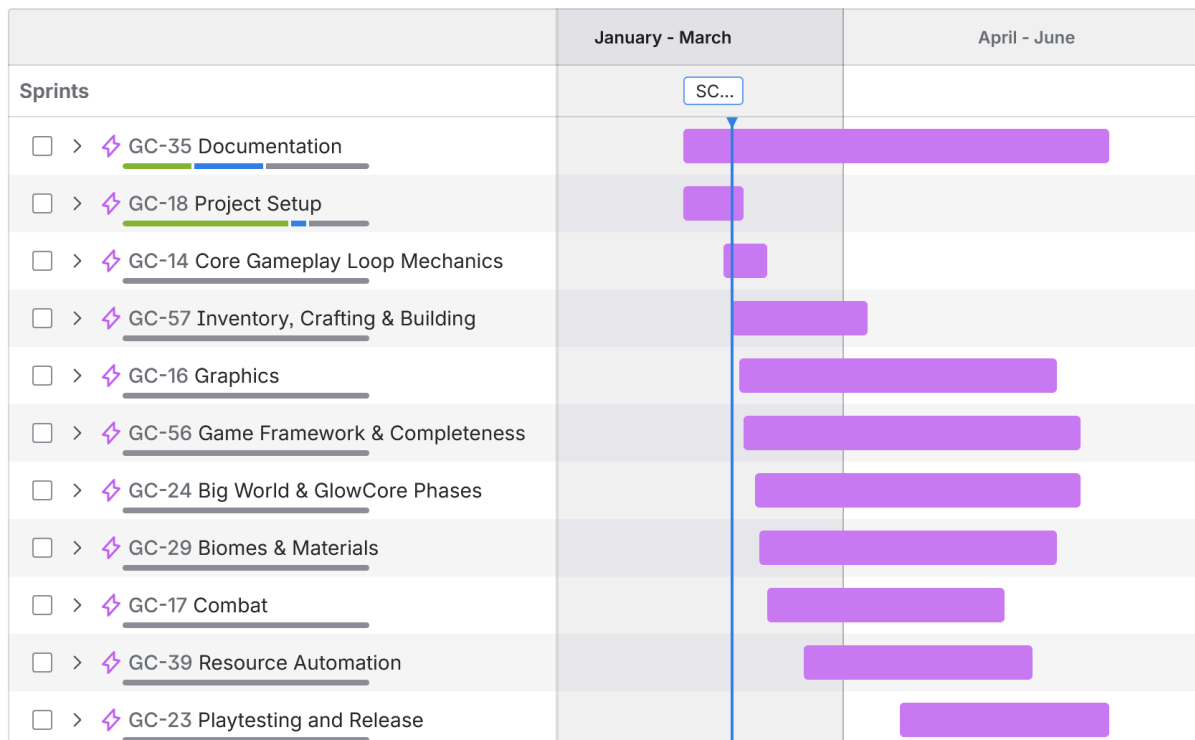


Image 1: Jira Timeline

6.3.2.3 Initial Backlog & MVP

The priority of Stories is defined by the ordering in the backlog. As we progress in Agile, we may divide Stories further and add new Stories. The MVP is reached when every Story up to and including **GC-101 Build World** is completed.

The following is the Initial Backlog as of 04.03.2026.

☐ Backlog (44 work items)				0 0 0	Create sprint
GC-54	There is a player	CORE GAMEPLAY LO...	TO DO	-	
GC-78	Create Tree-Node 3D Model	GRAPHICS	TO DO	-	
GC-95	Create Wood-Item 3D Model	GRAPHICS	TO DO	-	
GC-47	Trees can be chopped	CORE GAMEPLAY LO...	TO DO	-	
GC-109	Make GlowCore Phase 1 3D Model	GRAPHICS	TO DO	-	
GC-46	GlowCore expands World	CORE GAMEPLAY LO...	TO DO	-	
GC-69	Make Inventory-System	INVENTORY, CRAFTI...	TO DO	-	
GC-74	Save Game-Data	GAME FRAMEWORK &...	TO DO	-	
GC-108	Create Axe 3D Model	GRAPHICS	TO DO	-	
GC-97	Create Coal 3D Model	GRAPHICS	TO DO	-	
GC-106	Add Coal Node and Item	BIOMES & MATERIALS	TO DO	-	
GC-96	Create Stone 3D Model	GRAPHICS	TO DO	-	
GC-93	Add Stone Node and Item	BIOMES & MATERIALS	TO DO	-	
GC-107	Make Pickaxe 3D Model	GRAPHICS	TO DO	-	
GC-102	Add Pickaxe	BIOMES & MATERIALS	TO DO	-	
GC-79	Add Grassland Biome	BIOMES & MATERIALS	TO DO	-	
GC-111	Make GlowCore Phase 2 3D Model	GRAPHICS	TO DO	-	
GC-100	Add GlowCore Phase 2	BIG WORLD & GLOWC...	TO DO	-	
GC-71	Make Crafting-System	INVENTORY, CRAFTI...	TO DO	-	
GC-70	Make Building-System	INVENTORY, CRAFTI...	TO DO	-	
GC-77	Implement Pixelize-Shader	GRAPHICS	TO DO	-	
GC-103	Add Chest	INVENTORY, CRAFTI...	TO DO	-	
GC-101	Build World	BIG WORLD & GLOWC...	TO DO	-	
GC-99	Add Rock Biome	BIOMES & MATERIALS	TO DO	-	
GC-72	Game Intro	GAME FRAMEWORK &...	TO DO	-	
GC-73	Game Ending	GAME FRAMEWORK &...	TO DO	-	
GC-105	Add Combat Logic	COMBAT	TO DO	-	
GC-124	Create Enemy 3D Model	GRAPHICS	TO DO	-	
GC-81	Add Enemy	COMBAT	TO DO	-	
GC-75	Ingame Debug Settings	GAME FRAMEWORK &...	TO DO	-	
GC-76	Player Settings-Menu	GAME FRAMEWORK &...	TO DO	-	
GC-125	Create Peaceful-Mob 3D Model	GRAPHICS	TO DO	-	
GC-82	Add peaceful Mob	COMBAT	TO DO	-	
GC-123	Make Sword 3D Model	GRAPHICS	TO DO	-	
GC-83	Add Sword	COMBAT	TO DO	-	
GC-120	Make Axe-Machine 3D Model	GRAPHICS	TO DO	-	
GC-114	Add Axe-Machine	RESOURCE AUTOMAT...	TO DO	-	
GC-121	Make Replant-Machine 3D Model	GRAPHICS	TO DO	-	
GC-115	Add Replant-Machine	RESOURCE AUTOMAT...	TO DO	-	
GC-122	Make Rotater-Block 3D Model	GRAPHICS	TO DO	-	
GC-116	Add Rotater-Block	RESOURCE AUTOMAT...	TO DO	-	
GC-117	Conduct Playtesting with User	PLAYTESTING AND R...	TO DO	-	
GC-118	Release Game on Steam	PLAYTESTING AND R...	TO DO	-	
GC-119	Make Marketing Material for Steam Page	PLAYTESTING AND R...	TO DO	-	

Image 2: Inital Backlog

6.3.3 Short Term Plan

In a Sprint the priority of the tasks and stories is defined by the ordering.

6.3.3.1 Sprint 1 (Sprint Zero) 20.02.2026 - 06.03.2026

The following shows the Jira sprint backlog of the first sprint. Since this was Sprint Zero, not all guidelines were followed initially because they had not yet been established.

Work	Assignee	Status
 GG-88 Document Long and Short Term Plan	 Nathanael Fässler	DONE ▾
 GG-135 Document Sprint Planning Meeting Sprint 2	 Nathanael Fässler	DONE ▾
 GG-92 Setup Contract with OST about Ownership	 Nathanael Fässler	DONE ▾
 GG-89 Document Meeting Weekly Scrum Sprint 1	 Nathanael Fässler	DONE ▾
 GG-4 Concept for Code Quality, Linter etc.	 Yoris Kucera	DONE ▾
 GG-3 Unity Setup	 Yoris Kucera	DONE ▾
 GG-126 Documentation list of tables	 Cedric Cathomas	DONE ▾
 GG-1 Risk Specification	 Yoris Kucera	DONE ▾
 GG-91 Document Guidelines for code and documentation	 Yoris Kucera	DONE ▾
 GG-6 GitLab Timetracking Research	 Dominik Wyss	DONE ▾
 GG-5 GitLab Pipeline Setup	 Dominik Wyss	DONE ▾
 GG-2 GitHub Setup	 Yoris Kucera	DONE ▾
 GG-87 Document Meetings Structure	 Cedric Cathomas	DONE ▾
 GG-84 Typst documentation changes	 Cedric Cathomas	DONE ▾
 GG-86 Document Collaboration as a team	 Cedric Cathomas	DONE ▾
 GG-7 Typst Documentation Setup	 Cedric Cathomas	DONE ▾
 GG-8 People, Cost, Time Documentation	 Cedric Cathomas	DONE ▾
 GG-85 Roles Documentation	 Cedric Cathomas	DONE ▾
 GG-12 Longtermplan & Milestones	 Nathanael Fässler	DONE ▾
 GG-9 Jira Setup	 Nathanael Fässler	DONE ▾
 GG-112 Unity Folder Structure	 Nathanael Fässler	DONE ▾
 GG-11 Steamworks Registration	 Nathanael Fässler	DONE ▾
 GG-13 Shorttermplan for first sprint	 Nathanael Fässler	DONE ▾

Image 3: Sprint 1 Backlog

6.4 Risk Management

This chapter describes potential risks to the project and how we plan to mitigate them. We identified 8 risks, which are listed in the table below. Additionally, these risks are visualized in two risk matrices, one showing the initial risk assessment and the other showing the risk assessment after mitigation strategies are applied.

No.	Risk	Description	Mitigation Strategy
R01	Lack of familiarity with Unity	None of our team members have any significant experience with the Unity Engine. As such, this could heavily impact our development times.	We made ourselves familiar with the engine before the actual start of the project, to get an idea of the workflows.
R02	Unity CI/CD Complexity	We are depending on Unity's build system being robust enough to work in the context of a CI/CD pipeline. We also potentially rely on pre-existing GitHub Actions for Unity, which may not be well-maintained or compatible with our project.	We will set up the CI/CD pipeline very early into the project and did earlier tests. We tried out pre-existing GitHub Actions to make sure they satisfy our needs.
R03	Scope/Feature Creep	As we are developing a game, we run the risk of encountering scope creep, i.e. continuously planning new features, which would result in us eventually not being able to keep up with the workload.	We created a detailed project plan with long-term and short-term planning, along with well defined milestones and a clear MVP.
R04	Bad Code Quality	In large codebases it is easy to end up with badly designed systems that don't fit together with the rest of the code. This would hinder further development.	We do mandatory code reviews before changes can be merged. Additionally, we will set up CI/CD pipelines to automatically run code quality and formatting checks on pull requests.
R05	Team Member is MIA	A team member might be MIA for an extended period of time due to various reasons, such as an illness or damaged hardware. This would reduce our capacity and slow down development.	We can't reduce the possibility of this occurring, but we can distribute work across the remaining team members. We also avoid knowledge monopolies and always aim to share discoveries with the other team members.
R06	Personal Conflicts	There is always the potential for personal conflicts within the team, which would complicate development.	We openly communicate any interpersonal issues and try to resolve conflicts as early as possible.
R07	Data Loss	There is the potential of local storage media failing, resulting in significant data loss.	We mitigate this by frequently committing our changes to GitHub, and keeping untracked files in cloud based locations.
R08	Workload Underestimation	We might be underestimating the amount of work required for specific features and tasks.	We plan with a buffer and try to be conservative with our estimates. We also try to break down large tasks into smaller ones, which are easier to estimate.

Table 9: Risk Analysis

6.4.1 Risk Matrix

The below table shows the risk assessment of the identified risks, before any mitigation strategies are applied. The risks are categorized based on their probability of occurrence and their potential severity.

Probability	Severity			
	Negligible	Marginal	Critical	Catastrophic
Certain				
Likely			R04	R01
Possible		R02/R06		R03/R08
Unlikely				R05
Rare				R07
None				

Table 10: Risk Matrix

6.4.2 Risk Matrix (Mitigated)

The below table shows the risk assessment of the identified risks, after mitigation strategies have been applied.

Probability	Severity			
	Negligible	Marginal	Critical	Catastrophic
Certain				
Likely				
Possible				
Unlikely		R06	R01/R04/R05	R08
Rare	R07	R02		
None				R03

Table 11: Risk Matrix (Mitigated)

7 Time Tracking Report

7.1 Time Tracking

Every member tracks time on Jira on the corresponding task/story. Time spent in meetings is documented in the meeting protocols. The time is logged in 15 minute blocks in the format e.g. 3h 15m.

7.2 Time Statistics

The time is grouped by member and by sprint. Per sprint and person the time is rounded to the nearest hour.

Sprint	Cedric omas	Cath-	Nathanael Fässler	Yoris Kucera	Dominik Wyss	Total
1	23h		29h	21h	24h	97h
2						
3						
4						
5						
6						
7						
Total	23h		29h	21h	24h	97h

Table 12: Time Statistics Sprint 1

8 Personal Reports

9 Meeting Minutes

9.1 Weekly Scrum / 28.02.2026

Field	Value
Date	28.02.2026
Time	11:00–14:00
Sprint	Sprint 1
Location	Online
Participants	Nathanael Fässler, Dominik Wyss, Yoris Kucera
Excused	Cedric Cathomas
Absent	-
Meeting lead	Yoris Kucera
Minutes by	Nathanael Fässler
Links	-

9.1.1 Agenda

Agenda item	Owner	Result / Decision
GitHub Setup	Yoris	- We switched from GitLab to GitHub The following rules were discussed: - Pull Request requires one approval that is not the same person as the author. - Merging Methods are Merge and Squash. - Every Jira Task corresponds to one branch.
CICD Pipeline Setup	Dominik	- Web build and .exe build succeeded. - Pages Architecture with main and dev branches decided
Timetracking	Dominik	- We won't use GitLab timetracking because we switched to GitHub - Timetracking is in Jira: Everyone tracks time for individual Tasks/Stories.
Unity/Coding Setup	Yoris	Coding Guidelines decided
Steam Release	Nathanael	- Registered on Steamworks - Up to 30 days waiting time until application is processed - Maybe we need to make a contract with OST.
Testing	Nathanael	Unit-Test and Integration-Test Strategy discussed.
Long/Shorttermplan	Nathanael	- Hierarchy: There are Epics which contain Stories. - Everything that is not a game-feature is a task. - Longtermplan with epics according to the Jira Timeline - Shorttermplan per sprint on the Jira board

		• Priority is defined by the order in the backlog in Jira • MVP is defined • We will estimate Tasks/Stories with Story-points • On Jira we can create Bug-Tickets which we will not estimate with Storypoints
--	--	---

9.1.2 Todos

Todo	Owner	Due
Finish CICD Pipeline	Dominik	06.03.2026
Ask Advisor about contract	Nathanael	04.03.2026
Ask Advisor about project plan	Nathanael	04.03.2026

9.2 Weekly Scrum and Sprint Retrospective / 06.03.2026

Field	Value
Date	06.03.2026
Time	14:00-16:00
Sprint	Sprint 1
Location	Online
Participants	Nathanael Fässler, Dominik Wyss, Yoris Kucera, Cedric Cathomas
Excused	-
Absent	-
Meeting lead	Nathanael Fässler
Minutes by	Cedric Cathomas
Links	-

9.2.1 Agenda

Agenda item	Owner	Result / Decision
Board Review	All	- Walked through all current board tasks - Updated status on completed tasks
Done Tasks and New Functionalities	All	- Reviewed completed work from Sprint 1 - Discussed new functionalities and rules
Open Questions	All	Discussed open questions regarding project setup and workflow
Unity Graphics in Repository	Nathanael Fässler	Only .fbx files are to be committed to the repository for 3D graphics in Unity
Documentation Versioning	Cedric Cathomas	- Tags for documentation versions start with "d" (e.g. d1.1) - Version is updated automatically on pull request to main via GitHub Action
How do we manage smaller tasks that are not worth creating a new task on jira. (small documentation changes)	Dominik Wyss	We create ticket for small backlog tasks that do not need an own Jira item so we can reference it on our pull request and use the name of that ticket for the branch
Decide if we need a separate pdf for protocols	Cedric Cathomas	We decided that there's no use in having a separate page just for meeting minutes, so we have one thing less to worry about

9.2.2 Sprint 1 Retrospective

9.2.2.1 What Went Well

- Good and fast communication
- Good planning on Jira
- Good preplanning visible
- Product Owner did a good job
- Good and understandable selection of planning and working tools
- Pipeline and GitHub are working
- Typst is set up
- General setup went very well

9.2.2.2 What Didn't Go Well

- Problems with GitLab
- Didn't have enough time for some tasks

9.2.2.3 Improvements

- Better time management
- Better definitions of done
- Use GitHub from the beginning

9.2.3 Todos

Todo	Owner	Due
Create ticket for small backlog tasks that do not need an own Jira item	Nathanael Fässler	07.03.2026
Create all table references so documentation looks nice and page breaks are clean	Cedric Cathomas	07.03.2026
Adapt milestone dates to single dates (e.g. 16.02.2026 instead of a time frame)	Cedric Cathomas	07.03.2026
clean up documentation (spaces that are too big)	Cedric Cathomas	07.03.2026
Change date format in documentation to use dots	Cedric Cathomas	07.03.2026
Create task for combined build and documentation pipeline	Nathanael Fässler	07.03.2026
Use figures for images so they appear in the list of figures	Nathanael Fässler	07.03.2026
remove seperate meeting minutes pdf typst code	Cedric Cathomas	07.03.2026

Bibliography

List of Figures

Image 1 Jira Timeline 17

Image 2 Inital Backlog 18

Image 3 Sprint 1 Backlog 19

List of Tables

Table 1	Team availability for review meetings	9
Table 2	Project Management Roles	11
Table 3	Developer Roles	12
Table 4	Planned Meetings Overview	13
Table 5	Planning Tools	13
Table 6	Work Items	14
Table 7	Milestones	15
Table 8	Epics	16
Table 9	Risk Analysis	20
Table 10	Risk Matrix	21
Table 11	Risk Matrix (Mitigated)	21
Table 12	Time Statistics Sprint 1	22

Code Listings